Multiperspective Perceptron Predictor

Daniel A. Jiménez
Texas A&M University
and Barcelona Supercomputing Center
djimenez@acm.org

Abstract

The *multiperspective perceptron predictor* (MPP) is a branch predictor based on the idea of viewing branch history from multiple perspectives. The predictor is a hashed perceptron predictor using previous outcomes and addresses of branches organized in ways that go beyond traditional global and local history. MPP is combined with the winner of the previous CBP, TAGE-SC-L. This paper describes MPP, the combining procedure, and various optimizations. Performance and accuracy are quantified and the storage used is accounted. The CBP organizers' script reports the aggregate branch mispredictions per kiloinstruction as 3.3895 and the cycles on the wrong path per kiloinstruction as 144.699.

1 Introduction

Branch predictors typically use global and/or local (*i.e.* per-branch) history to match or find correlations between previous branch outcomes and the current branch. However, there are many other ways to organize history beyond global and local. For example, inspired by the work of Albericio *et al.* [3], Seznec *et al.* propose using the inner-most loop iteration counter (IMLI) [13] as a feature in a hashed perceptron predictor together with local and global history as an additional component of a TAGE-SC-L predictor.

This paper describes a hashed perceptron predictor [15] that incorporates many kinds of branch history to make a prediction. This *multiperspective perceptron predictor* (MPP) is combined with TAGE-SC-L to achieve good accuracy given a 192KB hardware budget. This paper updates the idea initially presented as the "multiperspective perceptron predictor" with and without TAGE in CBP 2016 [**? ?**].

2 Background and Related Work

The original perceptron branch predictor [9] used a table of perceptron weights vectors to predict the outcome of conditional branches by selecting a vector of small integer weights using the branch address, then finding the dot product of this vector with the global history in a bipolar representation. The weights were trained with perceptron learning: when the outcome of the branch matches a branch in the history, the corresponding weight is incremented, otherwise it is decremented, saturating at the maximum and minimum values for the bit width of the weights. A bias weight is also trained with the outcome of the branch without respect to any previous branch in the global history.

A number of improvements were proposed to improve accuracy and latency: ahead pipelining with path information [7], piecewise linear decision surfaces to get around the linear separability problem of perceptrons [8], and hashing history and path features to reduce the number of tables needed [10, 11, 15]. Perceptron-based branch prediction has gained considerable adoption in industry by companies such as AMD, Samsung, Oracle, IBM, and others [1, 5, 14] and other areas of microarchitectural prediction [2, 4, 16].

The hashed perceptron predictor [15] is similar to an idea of Loh and Jiménez called *modulo path-history* [10], while O-GEHL [11] is a very specific instance of the general technique. The idea is to have several tables, each indexed by a different hash of branch history. The tables have saturating confidence weights. The selected weights are summed and the prediction is taken if the sum is at least zero, not taken otherwise. On a mispredict or low-confidence correct prediction, the corresponding weights are incremented if the branch is taken, decremented otherwise. The hashed perceptron predictor, like modulo path-history and O-GEHL, improves over the original perceptron predictor [9] by breaking the one-to-one correspondence between weights and history bits, allowing a more efficient representation.

3 Multiperspective Perceptron Predictor

The multiperspective perceptron predictor (MPP) is a hashed perceptron predictor that uses several different kinds of control-flow history information to form hashes into tables, read out weights, sum them, and threshold the sum to make a prediction. The sum of weights is called y_{out} from the original perceptron paper. It is updated by incrementing or decrementing 6-bit saturating weights based on whether the branch is taken or not taken, respectively. The paper describes the features in detail. Training is done for mispredicted branches as well as for branches where the magnitude of the perceptron sum fails to exceed a training threshold, θ, that is set using a simplified version of Seznec's training algorithm from O-GEHL [11].

MPP uses a transfer function to improve accuracy by boosting values of more confident weights. The features and the transfer function were initially tuned using a genetic algorithm, then fine-tuned by making small random tweaks and hill-climbing.

The CBP 2025 simulator makes several predictions before doing an update, so speculative update of histories is highly recommended. To ease the burden on competitors, the organizers have provided a hook to speculatively update histories using ground truth taken/not taken information, so rollback to non-speculative history is never needed. They have also allowed an unlimited number of speculative history update entries to support this functionality.

MPP also speculatively updates the history tables and training threshold (θ). **However** this other speculative update is done using the combined MPP/TAGE-SC-L prediction, **not** the ground truth taken/not-taken. To be clear, predictor tables are updated speculatively but only using predictions, not using the actual branch outcomes.

'CBP 2025', June 21, 2025, Tokyo, Japan

Daniel A. Jiménez
Texas A&M University
and Barcelona Supercomputing Center
djimenez@acm.org

Because this speculative update uses predictions, the tables and θ may be updated with wrong information. The training process will undo the update when a speculatively updated but mispredicted branch is resolved. The predictor monitors when there are too many low-confidence branches in flight and puts MPP into a non-speculative update mode until some low-confidence branches commit.

To index the prediction tables, the hash value of a feature is computed using recent history information, hashed together with the address of the branch to be predicted, then taken modulo the size of the prediction table. The weights corresponding to the indices are read, summed, thresholded to make a prediction of taken or not taken. After exploring many organizations, I found the following features useful for branch prediction:

3.1 Traditional Features

The following features from traditional branch predictors are used:

GHIST. Global history is the outcome of a branch shifted into a large register, with 0 meaning not taken and 1 meaning taken. It is hashed by bitwise XORing multi-bit blocks of histories. Parameters to this feature give the starting and ending indices of the history register to hash. This feature is not actually used in the final configuration, but rather only used in combination with other histories as described below.

PATH. Path history is the sequence recent branch addresses. Addresses truncated to 16 bits are shifted into an array. Parameters include a depth, a shift. The array is hashed by accumulating a hash value up to the given depth.

LOCAL. Local history is a first-level table of per-branch shift registers selected by a hash of the branch address, then hashed as an index into a second level table of perceptron weights.

GHISTPATH. This is a combination of GHIST and PATH. The GHIST and PATH features are XORed together as they are computed.

BIAS. The value of this feature is 0. It will be XORed with the branch address to form an index into the table. This feature tracks the tendency of a branch to be taken or not, regardless of other branch history.

3.2 Novel Features

The following novel features are used:

IMLI. The innermost loop iteration counter [13] is used as a feature, but with an additional twist. In the original formulation, when a backward branch is encountered, the IMLI count is incremented if the branch is taken, otherwise it is reset capturing the behavior of loops with back-edges at the bottom produced by smart compilers. Some compilers are not smart or optimize for size, so I explore an alternate IMLI: when a forward branch is encountered, the IMLI count is incremented when the branch is not taken, and reset when it is taken, representing a loop exit. This represents the case where the loop exit is at the top of the loop body, followed by an unconditional jump at the bottom. Only the forward formulation IMLI turned out to be useful for MPP, probably because the backward IMLI is already incorporated into TAGE-SC-L.

MODHIST. Some (most) branches in the global history are not correlated to branch outcome, but a single bit different in two histories can lead to two different hash values, causing longer training times and increased aliasing pressure. Traditional (one-to-one) perceptron predictors can overcome this problem since they find correlations between individual branch outcomes and not hashed histories, they are still susceptible to what I call *branch misalignment*. Suppose a branch branches over another branch. The second branch sometimes appears in the branch history and sometimes does not appear, leading to the same branch outcomes appearing in different locations in the global history. Neither traditional nor hashed perceptrons, nor other hashed-based predictors such as TAGE, can handle this problem without expending additional table entries and increasing training time. I propose "modulo history" where only branches with addresses congruent to 0 modulo some modulus are recorded. Incongruent branches causing misalignment are filtered out of the history and ignored. The problem is that those filtered branches may indeed have some correlation; that correlation is hopefully captured by the other features. Modulo history has two parameters: the modulus (a small integer), and the history length.

MODPATH. Modulo path history is the same as modulo history, but uses branch addresses instead of outcomes. The parameters are the same and the hash is computed similarly to the PATH feature.

GHISTMODPATH. This feature combines modulo history with modulo path history in a way analogous to GHISTPATH above.

RECENCY. The predictor keeps a fixed-depth recency stack of recently encountered branches managed with least-recently-used replacement. The feature hashes the addresses in the stack. The parameters are the depth into the stack to hash, a shift by which to shift the accumulator after hashing.

RECENCYPOS. The depth in the recency stack where a branch address is encountered, or the fact that the branch is not present in the stack, have a surprising correlation with branch outcome. This feature has a single parameter: the depth into the stack to search. The hash value is the position where the address was found, or the maximum table index if the address was not found.

BLURRYPATH. Traditional branch path history is a precise record of the sequence of recent branches. "Blurry" path history records larger-granularity regions where branches have been encountered, and only shifts a region into the history when a new region is entered. Regions are computed as branch addresses right shifted by a certain amount given as a parameter. When a branch from a new region is encountered, the previous region is shifted into the history. The feature's parameters are the amount to shift, the depth within the history to hash, and a parameter that controls how much to shift each region number while generating the hash.

ACYCLIC. This feature keeps a history register H of length n and records the outcome of a branch with address PC in $H[PC(\mathrm{mod} n)]$. The intuition is that we would like to remove the effect of loops (*i.e.* cycles) in the history and just keep the most recent outcome of

any branch. Two kinds of acyclic history are used: one where H is an array of branch outcomes, and another where it is an array of hashed addresses of the corresponding branches. The parameters to the feature are the size of H and the amount by which to shift hashed elements of H.

BACKPATH. This feature is like PATH, but only records the history of backward branches.

TAGE. This feature incorporates the TAGE prediction and confidence shifted left by tuned parameters, allowing MPP to benefit from the cases where TAGE provides more accuracy than MPP alone.

3.3 Discussion of Features

None of the novel features are particularly good predictors by themselves. However, together with each other and with the traditional features, they provide alternate perspectives on branch history and allow finding new correlations. The features mentioned above are the ones that helped improve accuracy on the CBP 2025 workloads.

4 Optimizations

This section describe various optimizations.

Feature Selection. Feature selection was done using a genetic algorithm followed by a hill-climbing phase where hundreds of thousands of combinations of features were evaluated on the CBP 2025 traces. To reduce the computation time, I extracted traces from the CBP 2025 simulator into a bespoke format used for simulating only the branch predictor, eliminating the run-time overhead of simulating the rest of the machine. Once I settled on a good set of features I integrated the code into the CBP 2025 simulator. The best features I could find are given in the source code as an array of `history_spec` structs. A key aspect of the feature selection is that it was done with predictions that include TAGE-SC-L combined with MPP. Rather than optimizing MPP itself to minimize MPKI, the process trains MPP to find the best features such that the aggregate prediction of the combiner has the lowest MPKI. Since TAGE-SC-L already incorporates global history, the combined MPP's features have more of the modulo-history and other somewhat orthogonal features than a stand-alone version of MPP would.

Transfer Function. Multi-layer perceptrons use a non-linear transfer function between layers. It makes sense to try applying a transfer function to the perceptron weights before they are summed, since there may be a non-linear relationship between a weight value and the probability that a branch is taken. After much experimentation, I found that a transfer function of the form $\frac{ax}{1-bx^2}$ gave a significant improvement over the identity function. In particular, the function $\frac{3.2x}{1-\frac{x^2}{1600}}$ worked well. The function is represented as a lookup table indexed by the weight. Figure 1 shows the transfer function. The function has been tweaked slightly to improve accuracy. I applied the same optimization to the perceptron predictor (SC) in TAGE-SC-L. That lookup table ranges in value from -63 to 63 and may be found in the modified code for TAGE-SC-L.

Adaptive threshold training. Seznec found that good accuracy is achieved when the number of updates to due mispredictions

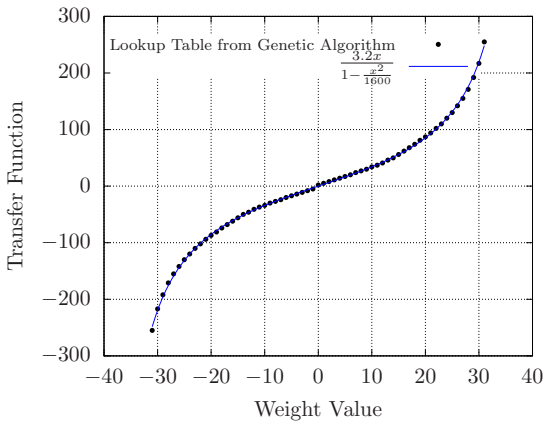

Figure 1: Transfer Function

is roughly the same as the number due to low-confidence predictions [11]. He gives an algorithm to adapt the threshold to maintain this property and I use that algorithm.

Extra information. Bits from the addresses of other control-flow instructions (*e.g.* unconditional branches, calls, and returns) are also considered in the branch history.

4.1 Making a Prediction

To make a prediction, the combiner (described later) checks to see if the branch has nontrivial behavior and should be filtered. Branches observed to be only taken or only not taken are predicted in that direction. Otherwise, MPP is used. Each feature with its parameters are used to hash the various histories together with the branch address to yield an index into that feature's table that selected a weight. The transfer function is applied to the weight and the results are summed. If the sum is at least zero, the branch is predicted taken, otherwise it is predicted not taken.

4.2 Updating the Predictor

When a nontrivial branch resolves, the algorithm decides whether to update the predictor. It uses the perceptron training rule to decide when to update. It must update the predictor for an incorrect prediction. If the prediction is correct, the magnitude of the perceptron output is compared to a threshold θ. If the magnitude is below θ then predictor is updated. To update the predictor, each weight that was used to make the prediction is incremented if the branch was taken or decremented otherwise. Weights are incremented or decremented with saturating arithmetic. A learning rate is applied to the perceptron output for threshold setting.

5 The Combiner

MPP is combined with TAGE-SC-L. The idea is to compute a linear combination of the perceptron confidence and the SC confidence. The slope and bias for the linear combination is tuned individually for each combination of: the TAGE-SC-L prediction (0 or 1), the MPP prediction (0 or 1), the TAGE(only) prediction (0 or 1), and the TAGE confidence(0, 1, or 2). There are $2 \times 2 \times 2 \times 3 = 24$ possible

Daniel A. Jiménez
Texas A&M University
and Barcelona Supercomputing Center
djimenez@acm.org

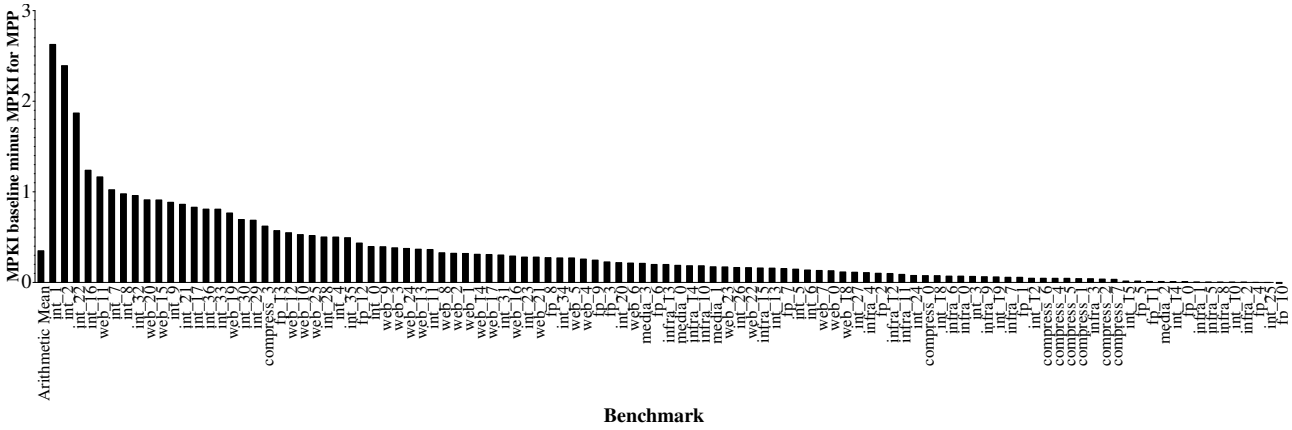

Figure 2: MPKI Difference

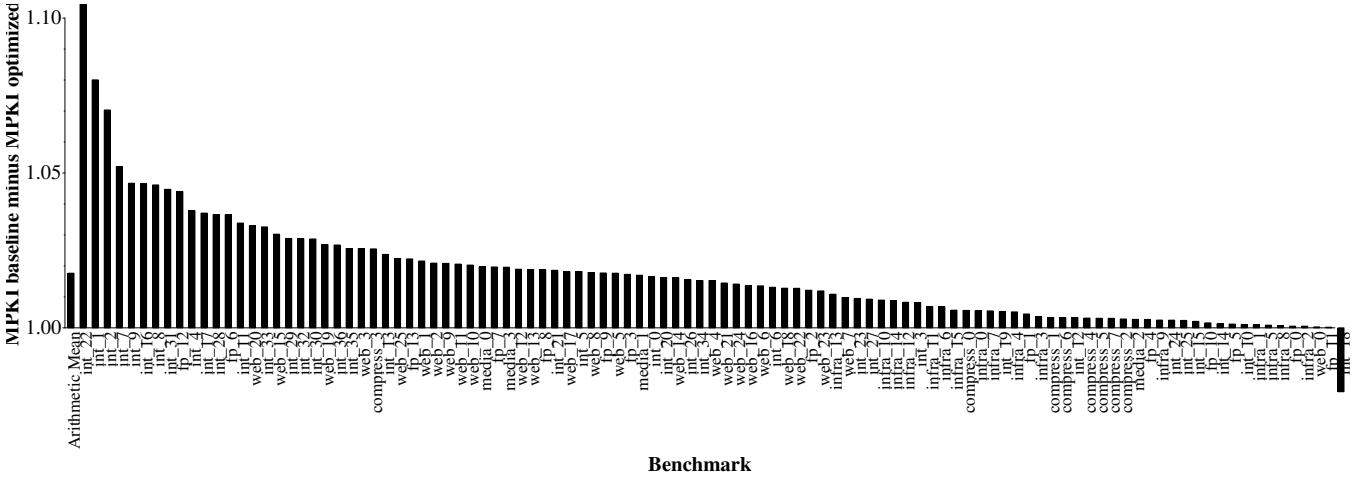

Figure 3: Speedup

combinations. I tuned most of them individually but some of them come up very infrequently and were not worth the computation so I tuned their slope and bias together.

Another bias is added to the sum, then thresholded to make a prediction. This bias is determined to minimize the recent number of misses. It is trained per combination so we keep track of 24 different biases ranging over 64 possible values of the bias. For each combination, the combiner keeps track of the number of misses each of the 64 possible bias values would have produced by counting up each time a value would have resulted in a miss and then decaying all the counters when one of them saturates at 7. Each counter requires 3 bits.

The combiner also provides filtering of trivial branches through two Bloom filters: one for taken branches and another for not taken branches. MPP will only train when a branch appears in both Bloom filters. Otherwise, the single observed behavior will be used to predict the branch. The first time a branch is encountered, it is predicted taken if the last 5 ghist bits are true, not taken otherwise. Most of the MPP histories are updated even for trivial branches based on a mask tuned per feature.

The Bloom filters add significant overhead (24KB) to the hardware budget. This would be unnecessary if the organizers of CBP 2025 would have provided access to a BTB.

In typical front-end designs, BTBs filter updates to predictor state using an "observed not taken" bit. A BTB entry is only allocated when a branch is taken. Thus, a branch can be filtered unless it hits in the BTB and has been observed not taken. A single extra bit per entry in an 8K-entry BTB could have served this purpose. Unfortunately, the organizers didn't provide BTB access, leading to the less realistic 24KB Bloom filter workaround. In fairness, some aspects of my predictor are a bit unrealistic too.

6 Results

Figure 2 shows, for each workload, the MPKI of the baseline figures provided by the CBP 2025 organizers minus the MPKI of my predictor. The arithmetic mean difference is 0.36 MPKI. The arithmetic mean MPKI of my predictor as reported by the simulation

infrastructure, per workload category, is 3.39. Figure 3 shows the speedup of my predictor over the baseline for each workload. The geometric mean speedup is 1.017. The aggregate "cycles on wrong path" for my predictor is 144.82.

6.1 Value of Optimizations

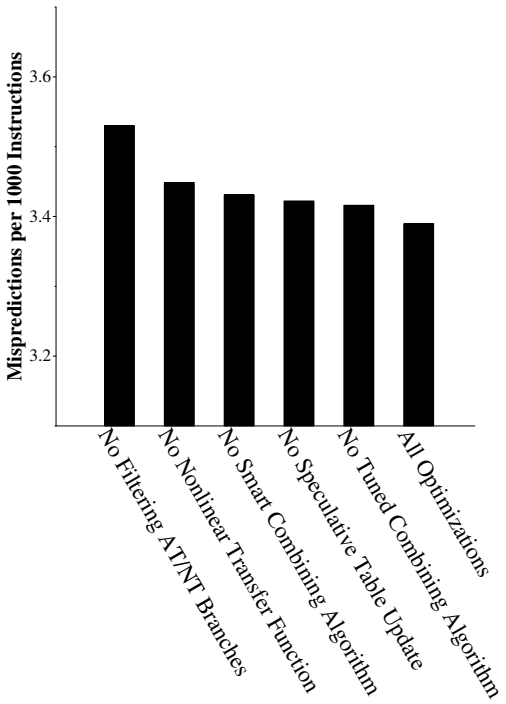

Figure 4: Value of Optimizations

Figure 4 quantifies the value of the major optimizations by independently removing optimizations and showing the resulting higher MPKI. Note that each bar represents keeping all the other optimizations except for one; the effect of removing combinations of optimizations is not illustrated. If applicable, the extra hardware taken by the missing optimization is used for extra perceptron weights. The most valuable optimization is filtering always/never taken branches. When these branches aren't filtered, MPKI goes up by 0.141, demonstrating the importance of shielding the perceptron predictor from destructive aliasing. The second most valuable optimization is the non-linear transfer function. When it is removed and replaced by a linear function that keeps the same range of y_{out} values, MPKI increases by 0.059. The third most valuable optimization is the combining algorithm that uses TAGE and MPP predictions and confidences with several tuned constants to make an aggregate prediction. When this optimization is replaced an obvious one that chooses the MPP prediction over TAGE only when the magnitude of y_{out} exceeds a constant multiple of θ (tuned to 0.5), MPKI is increased by 0.042. The next most helpful optimization is speculative update of weights tables using the prediction to guide perceptron learning and updating θ. Updating only non-speculatively increases MPKI by 0.033. The least helpful optimization is tuning the constants for the combining algorithm; when the perceptron and SC confidences are equally weighted and all linear functions are given a slope of 1 and a bias of 0, MPKI increases by 0.027.

7 Discussion

The predictor is somewhat complex and would be difficult to implement in hardware, but not impossible for a design team with resources and imagination. This championship does not take design complexity into account, so it would be unwise to consider realistic implementation as a first-class goal for my predictor. Some of the more challenging aspects of the predictor are as follows.

The computation has several sequential steps: a weight is read, then the transfer function applied, then summed with other weights, then thresholded. The latency could be mitigated with ahead pipelining [6, 12]. The only part of that computation not known (to me) to already be implemented in real branch predictors in industry is the lookup for the transfer function. This lookup could be taken off the critical path by implementing it during update rather than prediction by providing wider weights. The update would change a weight to the next value in the lookup table rather than simply incrementing by one. Multiple predictions per fetch block would require instantaneous speculative update of the global history. There are solutions in the literature, for example, rather than using taken/not-taken history, use bits from targets and/or branch addresses updated only on taken branches. The TAGE feature of MPP requires the output of the TAGE predictor, introducing more sequentiality. Again, this could be mitigated with ahead pipelining. The combiner requires keeping track of which of 64 values gives the best bias. The code does this with a frequent linear scan of the miss counter tables, but I am confident this hack could be avoided and the tables replaced with simple counters given enough thought. Local history is well known to be difficult to implement. There are several unsatisfactory solutions in the literature. Implementation requires a compromise between complexity and accuracy. Multiple speculatively updated features would require additional complexity to support rollback on misprediction recovery.

References

[1] Narasimha Adiga, James Bonanno, Adam Collura, Matthias Heizmann, Brian R. Prasky, and Anthony Saporito. 2020. The IBM z15 High Frequency Mainframe Branch Predictor Industrial Product. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. 27–39. https://doi.org/10.1109/ISCA45697.2020.00014

[2] Haitham Akkary, Srikanth T. Srinivasan, Rajendar Koltur, Yogesh Patil, and Wael Refaai. 2004. Perceptron-Based Branch Confidence Estimation. In *Proceedings of the 10th International Symposium on High Performance Computer Architecture (HPCA '04)*. IEEE Computer Society, USA, 265. https://doi.org/10.1109/HPCA.2004.10002

[3] Jorge Albericio, Joshua San Miguel, Natalie Enright Jerger, and Andreas Moshovos. 2014. Wormhole: Wisely Predicting Multidimensional Branches. In *Proceedings of the 47th Annual IEEE/ACM International Symposium on Microarchitecture* (Cambridge, United Kingdom) *(MICRO-47)*. IEEE Computer Society, Washington, DC, USA, 509–520. https://doi.org/10.1109/MICRO.2014.40

[4] Eshan Bhatia, Gino Chacon, Seth Pugsley, Elvira Teran, Paul V. Gratz, and Daniel A. Jiménez. 2019. Perceptron-Based Prefetch Filtering. In *2019 ACM/IEEE 46th Annual International Symposium on Computer Architecture (ISCA)*. 1–13.

[5] Brian Grayson, Jeff Rupley, Gerald Zuraski Zuraski, Eric Quinnell, Daniel A. Jiménez, Tarun Nakra, Paul Kitchin, Ryan Hensley, Edward Brekelbaum, Vikas Sinha, and Ankit Ghiya. 2020. Evolution of the Samsung Exynos CPU Microarchitecture. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. 40–51. https://doi.org/10.1109/ISCA45697.2020.00015

[6] Daniel A. Jiménez. 2002. Reconsidering Complex Branch Predictors. In *Proceedings of the 9th International Symposium on High Performance Computer Architecture (HPCA-9)*. 43–52.
[7] Daniel A. Jiménez. 2003. Fast Path-Based Neural Branch Prediction. In *Proceedings of the 36th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-36)*. IEEE Computer Society, 243–252.
[8] Daniel A. Jiménez. 2005. Piecewise Linear Branch Prediction. In *Proceedings of the 32nd Annual International Symposium on Computer Architecture (ISCA-32)*.
[9] Daniel A. Jiménez and Calvin Lin. 2001. Dynamic Branch Prediction with Perceptrons. In *Proceedings of the 7th International Symposium on High Performance Computer Architecture (HPCA-7)*. 197–206.
[10] Gabriel H. Loh and Daniel A. Jiménez. 2005. Reducing the Power and Complexity of Path-Based Neural Branch Prediction. In *Proceedings of the 2005 Workshop on Complexity-Effective Design (WCED'05)*. 28–35.
[11] André Seznec. 2005. Genesis of the O-GEHL Branch Predictor. *Journal of Instruction-Level Parallelism (JILP)* 7 (April 2005).
[12] André Seznec and Antony Fraboulet. 2003. Effective ahead Pipelining of Instruction Block Address Generation. In *Proceedings of the 30th International Symposium on Computer Architecture*. San Diego, California.
[13] André Seznec, Joshua San Miguel, and Jorge Albericio. 2015. The Inner Most Loop Iteration Counter: A New Dimension in Branch History. In *Proceedings of the 48th International Symposium on Microarchitecture* (Waikiki, Hawaii) *(MICRO-48)*. ACM, New York, NY, USA, 347–357. https://doi.org/10.1145/2830772.2830831
[14] Manish Shah, Robert Golla, Gregory Grohoski, Paul Jordan, Jama Barreh, Jeff Brooks, Mark Greenberg, Gideon Levinsky, Mark Luttrell, Christopher Olson, Zeid Samoail, Matt Smittle, and Tom Ziaja. 2012. Sparc T4: A Dynamically Threaded Server-on-a-Chip. *IEEE Micro* 32, 2 (2012), 8–19. https://doi.org/10.1109/MM.2012.1
[15] David Tarjan and Kevin Skadron. 2005. Merging Path and Gshare Indexing in Perceptron Branch Prediction. *ACM Trans. Archit. Code Optim.* 2, 3 (September 2005), 280–300. https://doi.org/10.1145/1089008.1089011
[16] Elvira Teran, Zhe Wang, and Daniel A. Jiménez. 2016. Perceptron learning for reuse prediction. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 1–12. https://doi.org/10.1109/MICRO.2016.7783705

A Cost Analysis

My predictor consumes 192KB. In keeping with past championships, I count only the mutable state of the predictor. Things like read-only lookup tables and the definitions of history features are part of the algorithm and not considered storage. The following items account for the 192KB hardware budget:

(1) I use the TAGE-SC-L implementation provided by the organizers. I assume this is 64KB (524,288 bits).
(2) There are 33 tables of 6-bit weights. The first table has 2,048 entries. Each of the remaining tables has 4,096 entries. This is a total of 133,120 entries, or 798,720 bits.
(3) There are 2 Bloom filters, each consisting of 3 tables of 32,768 bits each, for a total of 196,608 bits.
(4) There are 1,280 local history registers of 34 bits each, totaling 43,520 bits.
(5) There are 3,323 bits consumed by various global history, path history, modulo history, and other histories accounted in for detail in the code.
(6) There are 24 tables of 64 3-bit miss counters for a total of 4,608 bits.
(7) There is a 32-bit integer added to TAGE-SC-L to record TAGE predictions and confidence to communicate to MPP.
(8) The predictor records a number of bits to speculatively update histories. Organizers have allowed arbitrarily many copies of this sort of thing in a map, but we add the bits for a single entry to the hardware budget. The fields, from the combiner as well as MPP, are:
   (a) A 32 bit branch PC for the combiner
   (b) Two 64-bit pointers to prediction objects (would probably be smaller in hardware)
   (c) A 64 bit double precision float for the combiner sum
   (d) Another 32-bit PC for MPP
   (e) Two 16-bit hashes of the PC for MPP
   (f) A 32-bit integer for the value of y_{out}
   (g) 33 16-bit indices into perceptron weights tables
   (h) 1 bit to indicate a prediction has been updated
   (i) 1 bit to record the overall predictor of the combiner

   This structure sums to 850 bits.
(9) There are various counters and other miscellaneous items so I will add in 900 extra bits to avoid carefully litigating those values. That is an overestimate but I do not want to spend time on tiny details that could better be spent optimizing the predictor. I also note that the local histories of TAGE-SC-L and MPP are redundant and could be shared to slightly reduce storage overhead, but there are better uses of time than staring at André's code.

All of this adds to 1,572,849 bits, or 192KB.